

Predictive Autoscaling of Dockerized ML Workloads Using Prophet and LSTM

Ivan Bhargava[#], Nikunj Mahajan[#], Kanav Bhardwaj[#], and B. Thangaraju[#]

[#]*Department of Computer Science, International Institute of Information Technology (IIITB), Bangalore, India*

¹Ivan.Bhargava@iiitb.ac.in, ²Nikunj.068@iiitb.ac.in,

³Kanav.Bhardwaj@iiitb.ac.in, ⁴b.thangaraju@iiitb.ac.in

Received xx/xx/xxxx; Revised xx/xx/xxxx; Accepted xx/xx/xxxx

Abstract—Cloud-based machine learning (ML) workloads have unpredictable demand patterns. Traditional reactive autoscaling cannot meet these demands without risking Service Level Agreement (SLA) violations. This paper suggests a predictive autoscaling framework for Dockerized ML workloads. It combines Facebook Prophet and Long Short-Term Memory (LSTM) networks for demand forecasting.

We used 14-day synthetically generated workload traces which contained daily cycles, random bursts, and event flags. We compared five autoscaling policies based on metrics like under-provisioning events, cost index, and scaling switches. Prophet, along with a Buffered scaling policy, struck the best balance. It reduced SLA violations to 8 under-provisioning events at a cost index of 15,994, which is only 6% higher than the cheapest baseline forecast. Prophet outperformed LSTM with a Mean Absolute Error (MAE) of 3.57 and a Symmetric Mean Absolute Percentage Error (SMAPE) of 17.05%. This is because Prophet explicitly models seasonality and event factors. This work provides a reproducible, open-source approach for moving from forecasting research to practical autoscaling in containerized ML environments.

©2026 ICRAE. All rights reserved

Keywords—predictive autoscaling, Prophet, LSTM, Docker, cloud computing, machine learning, synthetic workloads, SLA violations, cost optimization.

I. INTRODUCTION

Cloud applications today work in very dynamic environments. User demand can change greatly due to the time of day, sudden events, or unpredictable factors. Static provisioning of compute resources, although simple, is inefficient because it either wastes capacity during quiet times or fails to meet Service Level Agreements (SLAs) in the event of sudden spikes.

Traditional mechanisms of autoscaling, which adjust resource allocations based on workload conditions, have become essential for elasticity in cloud systems. It typically relies on threshold-based rules or heuristic scaling policies, as noted in a 2014 review by Lorido-Botran et al. [1]. Though these methods avoid serious overloads, they are inherently reactive, i.e., they respond only after performance drop, which can be particularly harmful in latency-sensitive ML workloads running inside Docker containers [2].

In fact, even a short period of under-provisioning could lead to SLA violation and dissatisfied users. For example, by default the Kubernetes Metrics Server gathers resource-usage data every 60 seconds. When a defined threshold, such as CPU utilization, is crossed, the Horizontal Pod Autoscaler (HPA) usually responds within about 60 to 90 seconds to scale the workload up. Scaling down, however, is intentionally slower due to a built-in five-minute stabilization window designed to prevent rapid, unnecessary fluctuations.

Owing to these issues, predictive autoscaling has become a focus of investigation. Predictive methods can balance reliability with cost efficiency by anticipating demand in advance and pre-emptively adapting resources to that demand. Workload modeling and synthetic trace generation have been studied in order to evaluate strategies of resource allocation, such as in the CloudSim toolkit by Calheiros et al. (2011) [3] and in work by Sharma et al. (2011) [4]. Predictive techniques have also been investigated in domains such as smart grids, as reviewed by Zheng et al. (2020) [5], and smart manufacturing, as studied by Kim et al. (2018) [6]. These works demonstrated the superiority of forecast-driven resource management over static or reactive methods.

In this paper, we present a proof-of-concept framework for predictive autoscaling of Dockerized ML workloads. Our approach combines two different forecasting models: Prophet, a model detailed by Taylor and Letham (2018) [7], which is a statistically oriented model with seasonality and event effects; and Long Short-Term Memory (LSTM) networks, based on the 1997 work of Hochreiter and Schmidhuber [8], a type of neural architecture particularly well-suited for sequential data.

We perform a systematic evaluation of their performance across a wide range of autoscaling policies, from simple ceiling rules to buffered scaling, confidence aware scaling, and policy-aware incremental adjustments. Another important contribution of our work is the design of synthetic workload traces generated based on daily cycles, bursty spikes, random noise, and specific event markers. This controlled environment, inspired by established practices in systems research, lets us isolate the impacts of forecasting accuracy and policy design on system performance in a repeatable manner.

To support reproducibility and further research, the complete source code for this framework, including the synthetic data generators and policy simulators, is available in our GitHub repository [9]. Our results indicate that Prophet consistently outperforms LSTM for synthetic workloads and buffered scaling represents the best trade-off between cost and SLA conformance. This study tries to bring predictive autoscaling

from a merely theoretical concept to concrete real-world usage by combining current state-of-the-art machine learning forecasting techniques with an exhaustive assessment of autoscaling policies in a Dockerized ML environment.

Reactive scaling operates through threshold-based rules: if CPU exceeds 80%, add one container; if CPU drops below 30%, remove one container. This policy works only when the pressure is already visible. In contrast, predictive scaling uses historic trends and patterns, such as daily cycles or weekly peaks, to forecast future demand. This allows the advance scaling of containers, avoiding overload and unnecessary cost spikes, giving DevOps teams a way to proactively assign cloud resources, reducing SLA violations and cost overheads well before such problems might occur.

II. RELATED WORK

A. Cloud Autoscaling

Lorido-Botran et al. [1] surveyed autoscaling techniques and classified them into threshold-based, control-theory, reinforcement learning, and time-series-based approaches. They found that reactive threshold policies dominant in platforms such as AWS Auto Scaling and Kubernetes HPA, suffer from response lag that causes SLA violations under high traffic. Herbst et al. [10] described elasticity along three independent dimensions—provisioning speed, accuracy, and cost—providing a rigorous evaluation framework which is directly applicable to our policy comparison. Rossi et al. [11] compared reactive, proactive, and RL-based autoscalers on live Kubernetes clusters, finding that proactive methods consistently reduce tail latency under bursty loads.

B. Predictive Resource Management

Sharma et al. [4] demonstrated cost-aware elasticity by pre-provisioning resources based on workload forecasts, showing measurable improvements over reactive systems even with simple predictors. Bankole and Ajila [12] applied ARIMA-based prediction to cloud resource provisioning, achieving 10–20% cost savings over threshold-based methods on periodic workloads. These results motivate our use of more expressive forecasting models—Prophet and LSTM—to capture both seasonality and event-driven demand spikes.

C. Time Series Forecasting for Workloads

Taylor and Letham [7] introduced Prophet, which decomposes a time series into trend, seasonality, and holiday/event components with explicit changepoint handling, making it well-suited for cloud workloads with regular daily patterns and discrete events. Kumar and Tiwari [13] showed that LSTM networks capture nonlinear workload dependencies missed by statistical models, though they are prone to error accumulation in rolling multi-step forecasts—a limitation directly observed in our results. Zheng et al. [5] validated forecast-driven resource pre-allocation in smart-grid demand management, confirming that the approach generalizes beyond web workloads.

D. Container-based Scaling

Kim et al. [6] addressed predictive resource allocation for cloud-based manufacturing, confirming that forecast-driven policies outperform static provisioning. For ML inference specifically, Ali et al. [14] showed that inference latency is highly sensitive to under-provisioning due to nonlinear CPU saturation—a characteristic approximated by the stepped latency model used in our simulation. Our work extends these efforts by jointly evaluating two forecasting models across five autoscaling policies in a fully reproducible, open-source Dockerized ML simulation.

III. EXPERIMENTAL SETUP AND FRAMEWORK

A. Synthetic Data Generation

Real production workload traces were not available from any open-source sources. Hence, we generated synthetic demand data following parameters that come as close to realistic cloud traffic patterns as possible. The data is simulated at a 1-minute granularity for 14 days. The workload combines sinusoidal cycles to simulate diurnal peaks of usage (busy during the day and low at night), bursty spikes as short-lived surges inserted at random intervals representing flash events or sudden popularity spikes, random Gaussian noise representing unpredictable variability, and event flags as binary variables indicating whether something exceptional has occurred such as a promotion or an outage.

The combination of cycles, bursts, noise, and events creates a realistic trace that exhibits daily and weekly seasonality, with rare sharp bursts and stochastic background variation. The absence of publicly accessible production workload traces necessitated the use of synthetic datasets, a practice widely endorsed in prior research. Calheiros et al. [3] integrated synthetic workloads into CloudSim for evaluating resource-allocation methods, Sharma et al. [4] relied on them for workload modeling, and Zheng et al. [5] demonstrated their value in smart-grid forecasting. Accordingly, evaluation based on synthetic traces offers a credible proof-of-concept, with future validation on production traces expected to reinforce the results. Synthetic workloads allow for reproducible experiments that isolate the effects of forecasting accuracy from real-world noise and serve as a controlled baseline for comparing algorithms.

B. Split into Training and Testing Sets

The data is split chronologically to ensure a fair evaluation. The training set consists of the first 10 days used for model fitting, while the test set comprises the last 4 days kept for evaluation, which models did not see during training.

To visualize the architecture of our simulation, Figure 1 illustrates the complete end-to-end workflow, detailing how data flows from generation through to the final performance evaluation.

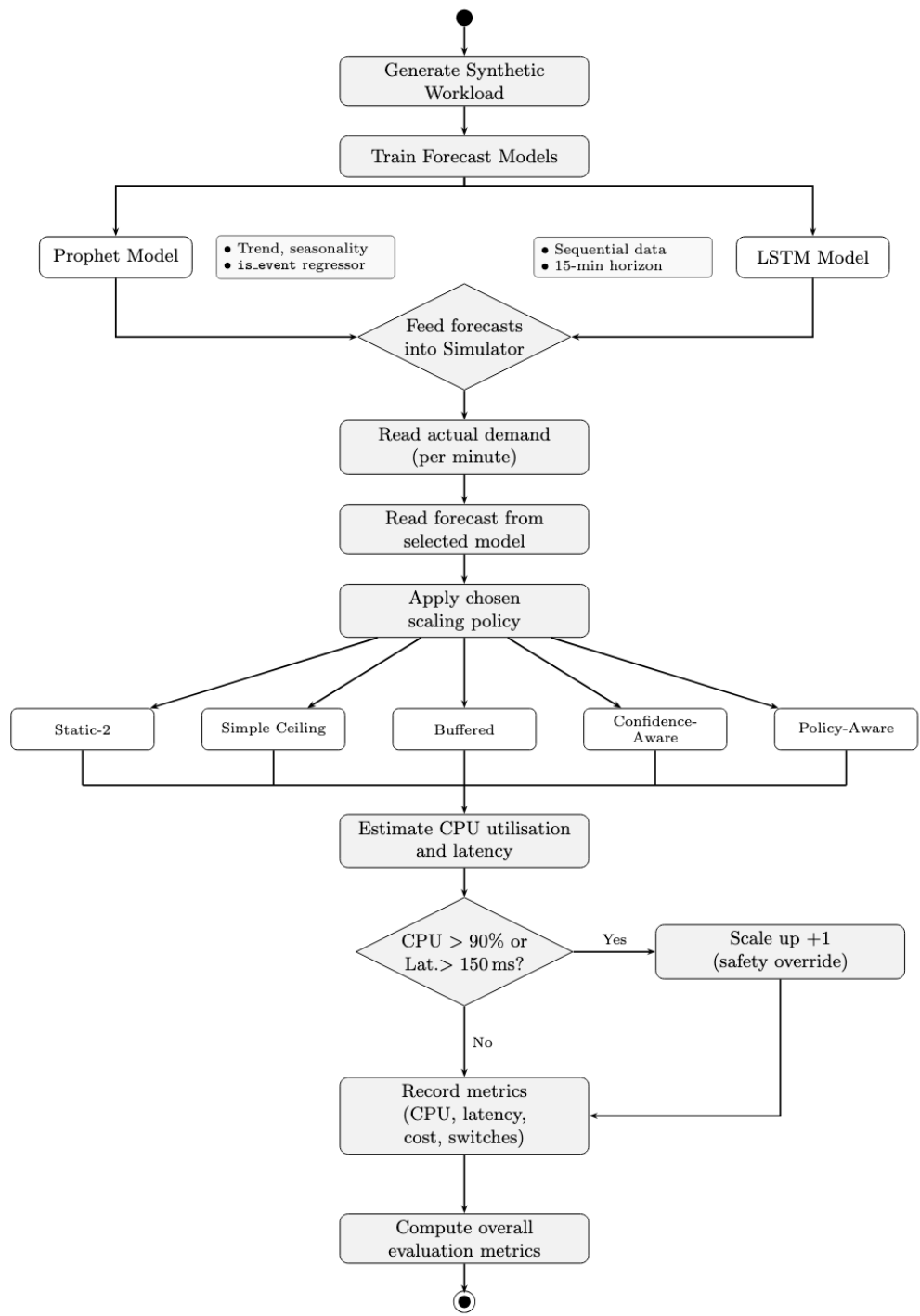


Figure 1: UML Diagram for the Experiment Setup

The main stages of the workflow of the system include the following four: (1) Synthetic Data Generation creates realistic workload traces, (2) Forecasting uses Prophet and LSTM to predict future demand, (3) Autoscaling simulation applies different scaling strategies, and (4) Performance Evaluation measures cost, latency, and SLA adherence. This structure made experiments repeatable and emphasized how forecast accuracy and policy design influence the outcomes.

IV. FORECASTING WITH PROPHET AND LSTM

A. Prophet: Forecasting Made Practical

Prophet (Facebook) [7] is a statistical time-series model which decomposes the demand into trend, seasonality, and external regressors. To capture the effect of specific events such as promotions or outages on demand, we utilize Prophet’s `add_regressor` functionality. The model is instantiated and dynamically configured with any external regressors present in the training data, in this case, the `is_event` flag prior to fitting. This allows the model to learn the magnitude of the demand spikes associated with these flags separately from the baseline trend and seasonality.

The implementation initializes a Prophet model, dynamically adds external regressors (such as 'is_event') which is crucial for the model to recognize sudden flagged events, then renames columns to Prophet's expected format (ds, y) and fits the model. Complete implementation details are available in the GitHub repository [9] in the file src/models/prophet_model.py.

B. LSTM: Learning from Sequences

LSTM [8] is a recurrent neural network specialized for sequential data. We have implemented the LSTM using PyTorch, adopting a sequence-to-one regressor design. The architecture processes a sliding window of historical data, 60 minutes long, through an LSTM layer. The last hidden state is then passed through a fully connected head (Linear) that predicts a single scalar value, representing the average demand for the upcoming 15-minute horizon. The LSTMRegressor class includes an LSTM layer processing the sequence and a fully connected head to map the final hidden state to a single demand value. The forward pass takes input of shape (Batch, Past_Window, Features), processes it through the LSTM, extracts the last time step's hidden state, and returns the prediction. Complete implementation details are available in the GitHub repository [9] in the file src/models/lstm_model.py.

Prophet's explicit seasonality modeling makes it particularly suited for workloads with regular patterns, while LSTM excels at capturing complex sequential dependencies when trained on sufficient historical data.

C. Mathematical Formulation

Workload Model. The synthetic demand signal $d(t)$ at minute t is defined as:

$$d(t) = \mu + A \sin\left(\frac{2\pi t}{T_d}\right) + \sum_k B_k e^{-\lambda(t-t_k)} \mathbf{1}[t \geq t_k] + \varepsilon(t) + \beta E(t) \quad (1)$$

where μ is the baseline demand, A is the diurnal amplitude, $T_d = 1440$ min is the daily period, B_k is the magnitude of burst k at time t_k , λ controls the burst decay rate, $\varepsilon(t) \sim \mathcal{N}(0, \sigma^2)$ is Gaussian noise, $E(t) \in \{0, 1\}$ is the binary event flag, and β is the event impact coefficient.

Prophet Model. Prophet fits an additive decomposition:

$$\hat{d}(t) = g(t) + s(t) + \beta_e E(t) + \varepsilon(t) \quad (2)$$

where $g(t)$ is a piecewise linear trend with learned changepoints. The seasonality component $s(t)$ is modeled as a Fourier series of order N :

$$s(t) = \sum_{n=1}^N \left[a_n \cos\left(\frac{2\pi n t}{P}\right) + b_n \sin\left(\frac{2\pi n t}{P}\right) \right] \quad (3)$$

with $P = 1440$ for daily seasonality. The term $\beta_e E(t)$ is the fitted event regressor, learned independently of trend and seasonality.

LSTM Model. The LSTM cell updates its hidden state via the following gate equations:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (4)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_C[h_{t-1}, x_t] + b_C) \quad (5)$$

$$h_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \odot \tanh(C_t) \quad (6)$$

where f_t , i_t , o_t are the forget, input, and output gates; $\odot$ denotes element-wise multiplication. A sliding window of $W = 60$ past time steps is used as input, and the forecast at horizon $H = 15$ min is given by $\hat{d}(t+H) = W_{\text{out}} h_T + b_{\text{out}}$.

Autoscaling Policies. Let $C = 25$ req/min denote per-container throughput. Container allocation $n(t)$ is computed as:

$$\text{Simple Ceiling: } n(t) = \lceil \hat{d}(t)/C \rceil \quad (7)$$

$$\text{Buffered: } n(t) = \lceil (1 + \alpha) \hat{d}(t)/C \rceil, \quad \alpha = 0.15 \quad (8)$$

$$\text{Confidence-Aware: } n(t) = \lceil \hat{d}_{\text{upper}}(t)/C \rceil \quad (9)$$

A safety override applies universally: if $\text{CPU}(t) > 90\%$ or $\text{latency} > 150$ ms, then $n(t) \leftarrow n(t) + 1$.

Performance Metrics. Forecasting accuracy is quantified by:

$$\text{MAE} = \frac{1}{T} \sum_{t=1}^T |d(t) - \hat{d}(t)|, \quad \text{SMAPE} = \frac{100}{T} \sum_{t=1}^T \frac{2|d(t) - \hat{d}(t)|}{|d(t)| + |\hat{d}(t)|} \quad (10)$$

The cost index is defined as the total container-minutes consumed: $\text{Cost} = \sum_{t=1}^T n(t)$.

V. POLICY COMPARISON AND RESULTS

The output of the forecasting was fed into a minute-by-minute simulator, which models autoscaling decisions. The simulation parameters include: throughput per container of 25 req/min, initial allocation of 2 containers at the beginning, cooldown of 10 minutes between scaling events, and SLA constraints with maximum average latency of 150 ms and maximum CPU utilization of 90%.

It will read the actual demand and forecast, apply a policy to compute the allocation, estimate CPU utilization, and assign latency based on load for each minute of the simulation. In the assignment of latency, it will be assigned as 40 ms if CPU is less than 80%, 150 ms if between 80-100%, and 2000 ms if demand is larger than capacity. A safety-net override scales up by one container if latency is greater than 150ms or CPU is greater than 90%.

TABLE I provides an overview of the five autoscaling strategies implemented in the simulation environment and summarizes the key characteristics of each approach.

TABLE 1: COMPARISON OF AUTOSCALING STRATEGIES AND THEIR CORE BEHAVIORS

Policy	Description	Behavior
Static-2	Fixed allocation of two containers	Stable but inefficient
Simple Ceiling	Forecast $\div$ capacity, rounded up	Low cost, reactive
Buffered	Forecast + 15% margin	Balanced and steady
Confidence-Aware	Uses Prophet’s upper confidence limit	Reliable but costly
Policy-Aware	Gradual up/down scaling	Fewer switches, slower to react

We measured both forecasting error and system-level performance. Forecasting metrics include MAE, RMSE, MAPE, and SMAPE. Autoscaling metrics include under-provisioning events (a count of specific moments where the actual demand exceeded our provisioned capacity, indicating we failed the users), over-provisioning percentage (the percentage of time CPU usage dropped below 40%, indicating we are wasting money on idle servers), switches (the total number of times we scaled up or down, as too many switches can destabilize the system), and cost index (a cumulative measure of container-minutes representing total uptime cost).

While most of the earlier works, such as Kim et al. [6], rely on benchmarking against static provisioning baselines, the Static-2 policy in our setup plays this role. This configuration underlines the shortcomings of the nonadaptive strategies. With these fixed benchmarks, our contribution extends to predictive autoscaling, where machine learning forecasts like Prophet and LSTM drive dynamic allocation policies.

A. Prophet-based Scaling

A comprehensive summary of the simulation metrics across all tested models and policies is presented in TABLE II.

TABLE 2: SIMULATION RESULTS — PROPHET AND LSTM ACROSS ALL POLICIES

Model	Policy	Over %	Under Events	Avg Lat. (ms)	Avg CPU %	Switches	Cost Index (container-mins)
Prophet	Static-2	99.62	38	40.45	42.74	1	20,160
Prophet	Simple Ceiling	93.01	181	40.05	42.27	91	15,084
Prophet	Buffered	98.36	8	40.00	39.60	91	15,994
Prophet	Confidence-Aware	99.99	0	40.00	12.26	521	96,276
Prophet	Policy-Aware	90.58	426	40.45	42.74	240	15,319
LSTM	Static-2	99.62	38	40.77	46.64	1	20,160
LSTM	Simple Ceiling	88.01	260	40.58	46.22	117	13,660
LSTM	Buffered	97.41	52	40.46	42.04	281	14,965
LSTM	Policy-Aware	85.65	480	40.77	46.64	177	13,730
LSTM	Confidence-Aware	— omitted: LSTM produces no native uncertainty estimates [†]					

[†]Confidence-Aware scaling requires per-step upper confidence bounds not available from LSTM.

This summary highlights that the Prophet-Buffered policy achieves the optimal balance for this workload. It successfully reduces under-provisioning events to single digits (8 events) while maintaining a cost index (15,994) that is only marginally higher than the simple reactive baseline, proving its efficiency. The specific impact of different autoscaling strategies using Prophet forecasts on system stability and cost is detailed in Figure 2.

Policy Comparison (Prophet, TEST)

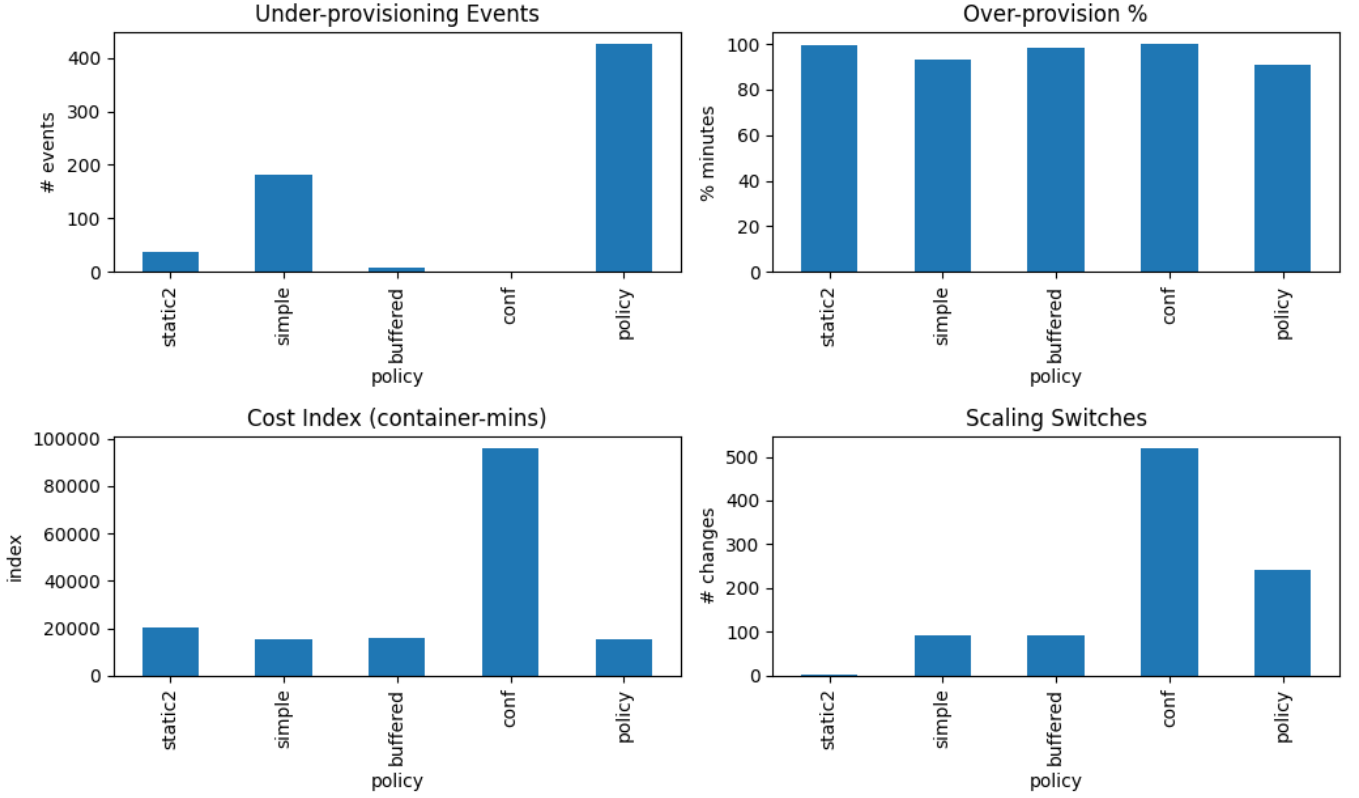


Figure 2: Performance comparison of autoscaling policies with Prophet: 3a. Under-provisioning: Minimal SLA violations due to Prophet’s robust handling of seasonal trends and demand spikes. 3b. Over-provisioning: High resource efficiency achieved by minimizing idle capacity through accurate trend decomposition. 3c. Cost Index: Optimized compute expenditure resulting from precise forecast-driven buffered scaling. 3d. Switches: Low frequency of allocation changes, reflecting superior system stability and reduced container thrashing.

Policies driven by the Prophet forecasts had a variety of results, as shown in Table II and Fig. 2. Static-2 exhibited high stability with 1 switch exercised but resulted in 38 under events and 99.6% over-provisioning, which pointed out the inefficiency of static allocation. Simple was inexpensive to produce, having a cost index of 15,084 but fragile, with 181 under events due to its inability to take into consideration forecast errors in periods of high demand. Buffered turned out to be the best-balanced policy, as adding a buffer of 15% trimmed under events down to just 8, with only a modest cost increase of 6% over Simple. Confidence-Aware completely eliminated under events (0) but at an impractical cost due to massive overprovisioning based on wide uncertainty intervals, with index 96,276. Policy-Aware, because the incremental logic trailed the demand bursts and hence had 426 under events, was under-responsive in its attempt to reduce thrashing.

B. LSTM-based Scaling

The downstream consequences of this forecast lag on system reliability are shown in Figure 3.

Policy Comparison (LSTM, TEST)

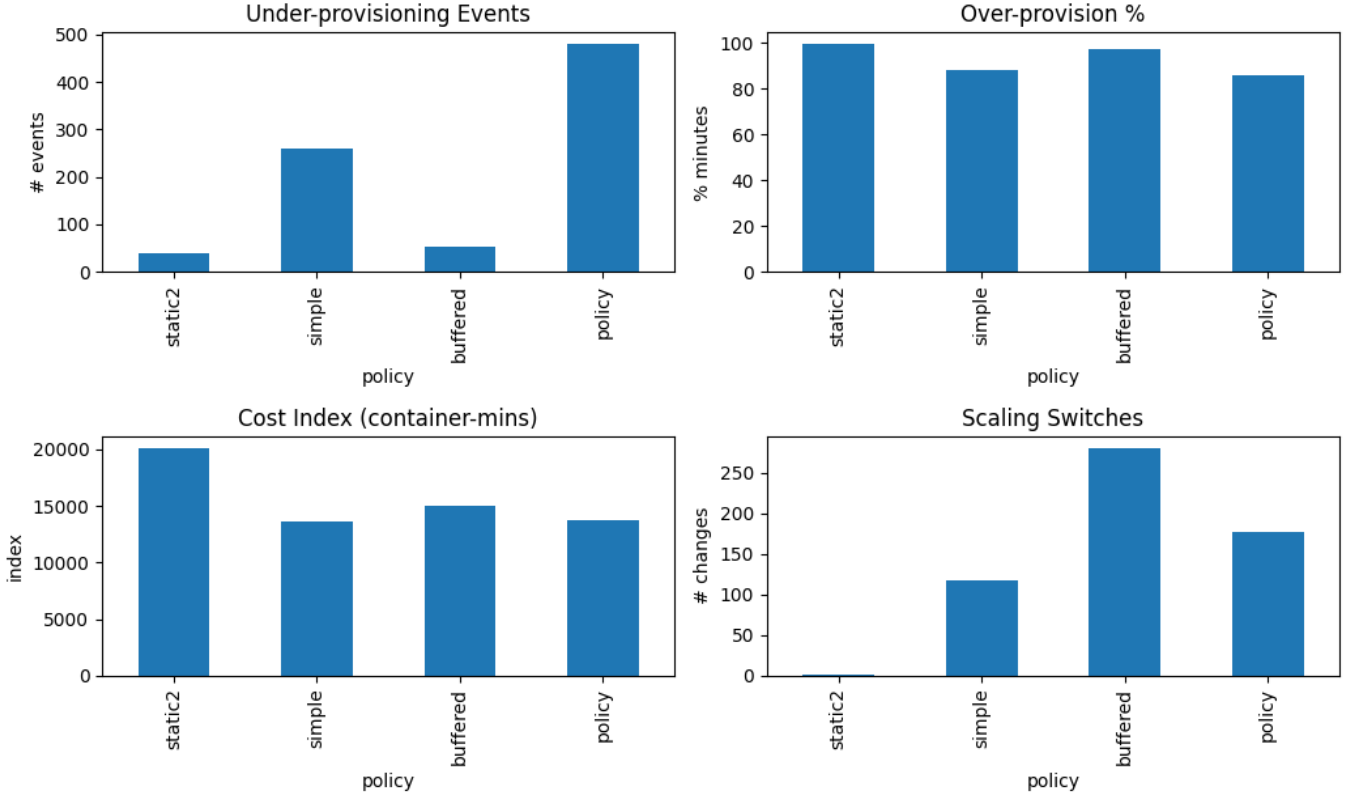


Figure 3: Performance comparison of autoscaling policies with LSTM: 4a. Under-provisioning: SLA violations caused by LSTM’s tendency to underpredict surges. 4b. Over-provisioning: Percentage of time resources remained underutilized. 4c. Cost Index: Cumulative resource expenditure for LSTM-driven scaling. 4d. Switches: Frequency of allocation changes, reflecting higher instability in LSTM results.

When driven by the LSTM forecasts, performance generally degraded (Fig. 3). Simple showed that the underprediction tendency of LSTM resulted in 260 under events, which demonstrates how forecast bias directly propagates into SLA violations. Buffered showed that the buffer reduced under events to 52, an improvement over the Simple but considerably worse than Prophet-Buffered that had 8, as the large errors by LSTM during surges exceeded the buffer margin. Policy-Aware had the worst combination of 480 under events since the slow scaling logic could not keep up with demand being already under-predicted. Effective autoscaling requires balancing multiple objectives: minimizing SLA violations while controlling costs and reducing allocation churn that can destabilize containerized environments.

VI. KEY FINDINGS AND PRACTICAL IMPLICATIONS

Prophet reported MAE = 3.57, RMSE = 5.47, MAPE = 20.96%, and SMAPE = 17.05% on the synthetic test data. These metrics confirm that Prophet captured both the dominant daily seasonality and the periodic bursts encoded in the workload. Including the event regressor further improved its handling of sharp spikes. In contrast, LSTM tends to be less stable. Because it predicts in a rolling one-step-ahead fashion, any small forecast error at time t propagates into the next prediction at $t + 1$, explaining the systematic underprediction observed in the test results, especially during demand surges.

Fig. 4 depicts the forecasting performance of the Prophet model overlaid against the actual ground truth demand.

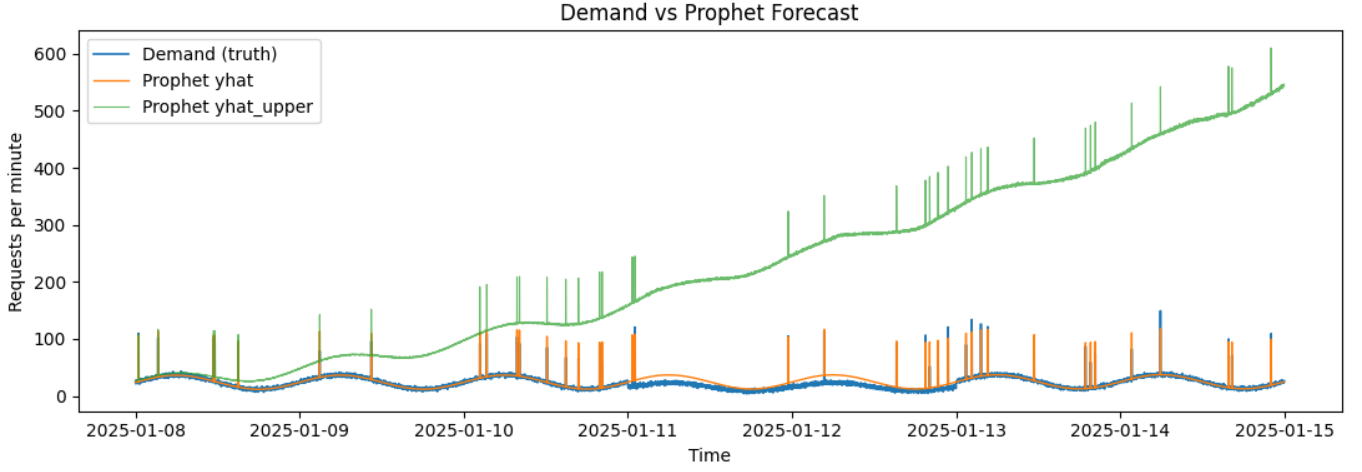


Figure 4: Prophet model’s forecast (yellow) line against the truth demand (blue).

Prophet is to an extent correctly predicting the spikes. In contrast to the statistical model, Figure 5 illustrates the forecasting behavior of the LSTM network on the same test data.

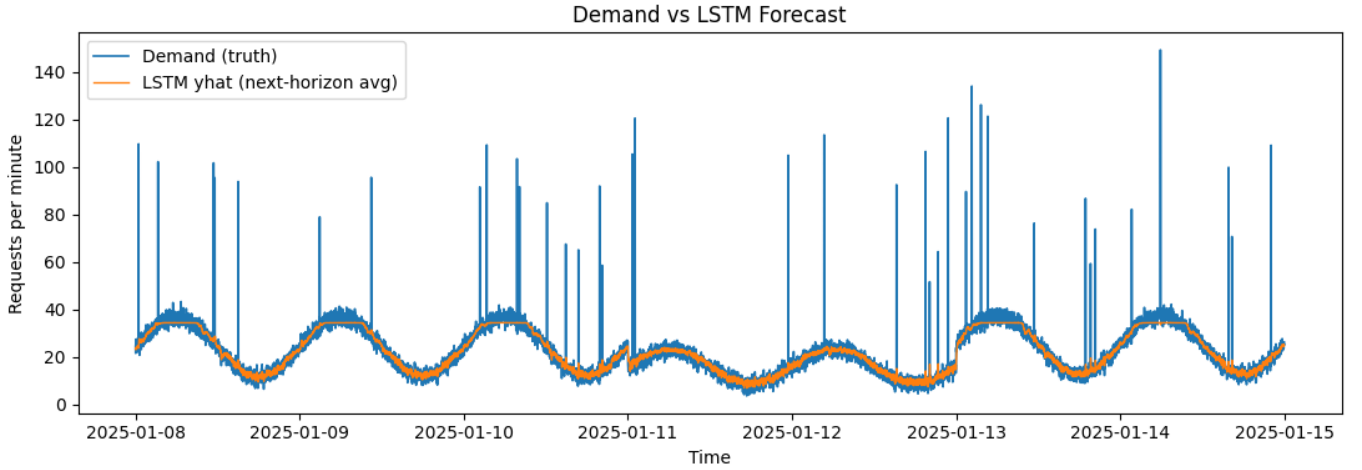


Figure 5: Demand (blue) vs LSTM model’s forecast (yellow). LSTM is not able to predict spikes in demand.

Fig. 4 and Fig. 5 show that Prophet outperformed LSTM because it has a structural advantage to model seasonality and events that exactly matched the design of synthetic data, whereas the latter suffered from error accumulation in rolling forecasts. The Buffered policy demonstrated superior performance across metrics. Forecast accuracy directly determines scaling effectiveness: Prophet’s 17% SMAPE enabled near-optimal resource allocation, while LSTM’s systematic underprediction during spikes led to persistent SLA violations despite buffering strategies.

VII. FROM SIMULATION TO REAL DEPLOYMENT

While these results confirm the promise of predictive autoscaling, this work represents a Phase-1 proof of concept. The most obvious next step is to evaluate this framework using real workload traces, to establish whether the robustness seen here can persist under real-world conditions. Following this, we plan to deploy the framework in a real containerized deployment and analyze its performance using live traffic. This will bridge the gap from simulation to practical cloud operations. Finally, complementing the work of Kim et al. [6], we would like to extend this methodology to other areas that are intensively dependent on demand forecasting, such as financial systems, e-commerce, and edge computing.

VIII. CONCLUSION

This work presented a proof-of-concept framework for predictive autoscaling of Dockerized ML applications. Building on prior work such as Kim et al. [6], we extended this approach to include state-of-the-art forecasting techniques, namely Prophet and LSTM, combined with various

scaling policies. Our results point to two main observations. Forecast accuracy has a direct impact on scaling behavior, and Prophet, by explicitly modeling seasonality, performed better than the LSTM on our synthetic workloads. Policy choice also matters: the Buffered strategy offered the best balance, reducing SLA violations without excessive resource use. The combination of Prophet with Buffered scaling delivered near-perfect SLA compliance and clearly outperformed the static policies. A distinctive aspect of this work is the combination of machine learning-based forecasting together with the systematic evaluation of multiple autoscaling policies. By designing a controlled synthetic workload, we enable a repeatable and transparent testbed for benchmarking scaling policies. This dual focus on both these core elements positions this approach highly flexibly for future research.

REFERENCES

- [1] T. Llorido-Botran, J. Miguel-Alonso, and J. A. Lozano, "A Review of Auto-scaling Techniques for Elastic Applications in Cloud Environments," *Journal of Grid Computing*, vol. 12, no. 4, pp. 559–592, 2014.
- [2] Docker Inc., "Docker: Lightweight Linux containers for consistent development and deployment," 2013. [Online]. Available: <https://www.docker.com>
- [3] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. De Rose, and R. Buyya, "CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," *Software: Practice and Experience*, vol. 41, no. 1, pp. 23–50, 2011.
- [4] U. Sharma, P. Shenoy, S. Sahu, and A. Shaikh, "A cost-aware elasticity provisioning system for the cloud," in *Proc. 2011 IEEE 31st Int. Conf. Distributed Computing Systems*, 2011, pp. 559–570.
- [5] Z. Zheng, Y. Yang, J. Chen, A. Bouguettaya, and B. Benatallah, "Forecasting in smart grids: An overview," *IEEE Trans. Computer Science and Engineering*, vol. 7, no. 2, pp. 1–16, 2020.
- [6] H. Kim, M. Kang, S. Park, and J. Choi, "Resource allocation for cloud-based smart manufacturing platforms," *Int. J. Precision Engineering and Manufacturing–Green Technology*, vol. 5, no. 4, pp. 519–530, 2018.
- [7] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
- [8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [9] Predictive Auto-Scaling of Dockerized ML Applications. Available: <https://github.com/ivan825/predictive-auto-scaling-of-dockerized-ml-applications>
- [10] N. R. Herbst, S. Kounev, and R. Reussner, "Elasticity in cloud computing: What it is, and what it is not," in *Proc. 10th Int. Conf. Autonomic Computing (ICAC)*, 2013, pp. 23–27.
- [11] F. Rossi, V. Cardellini, and F. Lo Presti, "Hierarchical scaling of microservices in Kubernetes," in *Proc. IEEE Int. Conf. Autonomic Computing and Self-Organizing Systems (ACSOS)*, 2020, pp. 1–10.
- [12] A. A. Bankole and S. A. Ajila, "Cloud client prediction models for cloud resource provisioning in a SaaS environment," in *Proc. IEEE 7th Int. Symp. Service-Oriented System Engineering (SOSE)*, 2013, pp. 156–161.
- [13] R. Kumar and M. Tiwari, "Cloud workload prediction using LSTM-based deep learning," *J. Cloud Computing*, vol. 9, no. 1, pp. 1–18, 2020.
- [14] M. Ali, R. Pincioli, F. Yan, and E. Smirni, "NESTO: An edge server selection framework for online inference in mobile augmented reality," in *Proc. IEEE/ACM Int. Conf. Utility and Cloud Computing (UCC)*, 2020, pp. 65–74.